Space Empire — Jeu Web

Document de conception technique

Mars 2026

1. Logiques et choix de conception

1.1 Stack technique

Le projet est développé avec la stack suivante :

- Symfony (framework PHP principal)
- RabbitMQ — gestion des événements différés (arrivée de flotte, fin de construction, etc.)
- Mercure — notifications temps réel vers le navigateur du joueur
- MariaDB + Doctrine ORM — persistance de l'état du jeu
- Docker — conteneurisation de l'ensemble des services

> *Le calcul du combat reste en PHP dans un service Symfony dédié. Un micro-service Python séparé a été évalué mais rejeté : le système de combat est déterministe et ne requiert pas de calcul intensif. Ajouter un container Python introduirait de la complexité réseau sans bénéfice réel à ce stade.*

1.2 Modèle de temps — pseudo temps réel

Le jeu ne fonctionne pas en temps réel strict. Le modèle retenu est le suivant :

- Chaque action planifiée (envoi de flotte, construction, recherche) génère un message RabbitMQ avec un délai (TTL ou delayed exchange).
- À l'échéance, le consumer RabbitMQ traite l'événement : résolution du combat, livraison des vaisseaux, fin de recherche, etc.
- Mercure notifie le joueur concerné dès que l'événement est traité.

Ressources — lazy calculation

La production de ressources n'est pas calculée à intervalle fixe. On stocke :

- current_stock : stock actuel au dernier calcul
- production_rate : taux de production par seconde
- last_updated_at : horodatage du dernier calcul

Le stock réel est calculé à la demande :

```
stock_réel = current_stock + (production_rate × secondes_écoulées)
```

La persistance n'a lieu que lors d'une action qui consomme des ressources (lancement de construction, envoi de flotte, etc.). L'interface affiche une estimation JavaScript côté client à titre informatif entre deux requêtes.

1.3 Carte spatiale

- La carte est générée en amont et chargée au démarrage du serveur. Elle ne change pas pendant la partie.
- Structure : Univers → Galaxies → Systèmes stellaires → Planètes (1 à 9 par système).
- Les joueurs connaissent le nombre de galaxies et de systèmes stellaires, mais pas le nombre de planètes par système.
- Pour découvrir les planètes d'un système, le joueur doit envoyer une sonde. Une sonde découvre toutes les planètes du système en une seule mission.
- La visibilité est gérée via une table PlayerDiscovery (Fog of War) : chaque système stellaire découvert par le joueur y est enregistrée.

1.4 Planètes et ressources

- Chaque planète a une position (1 à 9) par rapport à son étoile.
- La position influe sur les taux de production de base :
  - Position proche de 1 (proche de l'étoile) : bonus métal/carbone/énergie, malus hydrogène.
  - Position proche de 9 (loin de l'étoile) : inverse.
- Les ressources sont stockées par planète, pas globalement par joueur.
- Les ressources disponibles sont : métal, énergie, antimatière, hydrogène, carbone. D'autres pourront être ajoutées.
- Le transport inter-planètes se fait via des flottes équipées de vaisseaux cargo.

1.5 Bâtiments

- Chaque bâtiment appartient à une planète.
- Un bâtiment ne peut être construit qu'une seule fois par planète, mais son niveau peut augmenter.
- La liste des types de bâtiments est définie dans des fichiers YAML (catalogue statique).

Slug	Nom	Rôle
mine	Mine / Extracteur	Production de ressources (un type par mine)
storage	Stockage	Capacité de stockage par type de ressource
lab	Laboratoire	Permet de lancer des recherches sur la planète
shipyard	Chantier spatial	Construction de vaisseaux
fortress	Forteresse	Construction de défenses statiques

- Les prérequis de construction varient par palier de niveau (ex : niveau 1 sans prérequis, niveau 3 requiert telle technologie).
- Le chantier spatial ne peut pas être dupliqué sur une planète, mais son niveau réduit le temps de construction des vaisseaux.

1.6 Vaisseaux

- Les vaisseaux sont définis dans des fichiers YAML. Ce sont des données statiques de catalogue.
- Il n'y a pas de notion de niveau sur un vaisseau. La recherche débloque des vaisseaux plus performants (ex : chasseur → chasseur lourd → chasseur furtif).
- Une flotte peut contenir des vaisseaux de types différents (ex : 5 chasseurs + 5 intercepteurs).
- Les vaisseaux construits ne peuvent pas être améliorés ni recyclés.

Catégorie à redéfinir

Catégorie	Vaisseaux	Rôle principal
Ailes légères	Chasseur, Intercepteur, Chasseur lourd	Anti-bombardier, protection de flotte
Bombardier	Bombardier	Anti-gros / anti-défenses statiques
Escorte	Corvette, Frégate, Destroyer	Contrôle des formats légers
Ligne lourde	Croiseur, Cuirassé	Briser l'escorte, duels de ligne
Super-lourd	Capital Ship	Apex de projection, domine la ligne lourde
Support	Transport, Sonde, Vaisseau colonie	Logistique, exploration, colonisation

1.7 Construction de vaisseaux

- La construction se fait planète par planète. Chaque planète avec un chantier spatial a sa propre file de construction.
- Un seul chantier par planète. La construction en parallèle est possible sur des planètes différentes.
- Temps de construction : linéaire (1 vaisseau = 1 min → 10 vaisseaux = 10 min). Réduit par le niveau du chantier et les technologies.
- La file est stockée en BDD (ShipyardQueue), un message RabbitMQ est émis à chaque ajout avec le délai correspondant.
- À la fin de la construction, les vaisseaux sont ajoutés à la flotte de la planète. Si une ligne FleetShip avec ce slug existe déjà, on incrémente la quantité.

1.8 Flottes

- Une flotte appartient à un joueur, pas à une planète. La planète d'origine est mémorisée pour le retour.
- Statuts possibles : idle, traveling, returning, in_combat.
- Une flotte peut contenir plusieurs types de vaisseaux (plusieurs lignes FleetShip avec des slugs différents).
- Le transport inter-planètes utilise exactement le même mécanisme qu'une flotte de combat.
- Une sonde est un vaisseau comme les autres (construite, envoyée, détruite à l'arrivée). Elle découvre toutes les planètes du système cible.
  - Il y a deux types de sondes :

    Espionnage => espionne un planète en particulier

    Exploration => découvre les planètes d'un systèmes
- La colonisation consomme le vaisseau colonie à l'arrivée.

1.9 Système de recherche

- 1 recherche active maximum par planète (laboratoire requis).
- Si une technologie est en cours ou en queue sur une planète, elle ne peut pas être lancée sur une autre planète simultanément.
- Les niveaux de technologie sont globaux au joueur (communs à toutes ses planètes).
- Le niveau du laboratoire est local à la planète et influe uniquement sur la vitesse de recherche et les items à rechercher (indirect via les prérequis du yaml).
- Les technologies ont deux rôles :
  - Déblocage : prérequis pour construire un vaisseau, un bâtiment, ou accéder à une autre technologie.
  - Amélioration de performance : bonus de stats (précision, tracking, etc.) appliqués à la flotte du joueur.

> *Les bonus technologiques sont calculés au moment du lancement du combat (quand la flotte arrive à destination), pas au moment de l'envoi de la flotte. Une technologie terminée pendant le trajet est donc prise en compte. Une technologie terminée pendant le combat lui-même ne l'est pas.*

1.10 Catalogues statiques en YAML

Tout ce qui est données de game design (stats des vaisseaux, coûts, formules de scaling des bâtiments, arbre technologique, matrice de combat) est stocké dans des fichiers YAML versionés dans le code source, pas en base de données.

Avantages :

- Historique complet des modifications via Git (qui a changé quoi, quand, pourquoi).

- Impossible de modifier silencieusement une stat en production sans commit visible.
- Pas de migration de BDD pour un simple nerf ou buff.

Ce qui reste en BDD (état dynamique du jeu) :

- Ce que le joueur possède (vaisseaux, bâtiments, technologies débloquées).
- Les niveaux actuels des bâtiments et technologies.
- Les stocks de ressources et les files de construction/recherche.
- Les flottes et leur état.

> *Quand un YAML est modifié (nerf/buff), les effets s'appliquent immédiatement à tous les calculs futurs. Les vaisseaux déjà construits utilisent les nouvelles stats. Ce comportement est volontaire et documenté.*

1.11 Système de prérequis unifié

Vaisseaux, bâtiments et défenses partagent le même format de prérequis dans leurs YAML :

- technologies : liste de { slug, level } — prérequis global au joueur.
- buildings : liste de { slug, level } — prérequis local à la planète.
- ships : liste de slugs — le joueur doit avoir « débloqué » ce vaisseau (ses propres prérequis sont satisfaits récursivement).

Pour les bâtiments, les prérequis sont définis par palier de niveau. Pour construire le niveau N, on applique les prérequis du palier inférieur ou égal le plus proche.

> *Un vaisseau est « débloqué » si tous ses unlock_requirements sont satisfaits, évalués récursivement. Il n'est pas nécessaire d'en avoir construit un — il suffit d'en avoir la capacité.*

1.12 Système de combat

Le combat est calculé entièrement côté serveur au moment de l'arrivée de la flotte. L'issue est connue immédiatement. Les logs sont ensuite rejoués côté client pour l'affichage progressif.

Déroulement

- Le combat se déroule en rounds (nombre maximum défini, au-delà la flotte attaquante bat en retraite).
- Chaque vaisseau est une entité individuelle dans le resolver (pas un pool).
- Au début de chaque round, chaque vaisseau choisit sa cible parmi les ennemis (probabiliste).
- Un vaisseau peut être détruit avant d'attaquer dans le même round.
- Chaque vaisseau effectue un nombre de tirs aléatoire (entre 1 et son max).

- À la fin du combat, les survivants retrouvent leurs HP complets (pas de HP persistant entre combats).

Formule de toucher

```
Chance de toucher = clamp(15 ; 95 ; Précision + Tracking - Esquive_cible +
(Signature_cible / 5) - 15)
```

Paramètres des vaisseaux impliqués :

Stat	Rôle
precision	Capacité générale à toucher
tracking	Capacité à suivre les cibles agiles (anti-légères)
evasion	Capacité de la cible à esquiver
signature	Facilité à être touché (gros/lent = signature élevée)

Modificateurs de dégâts (matchup)

```
Dégâts finaux = Attaque de base × modificateur de contre
```

Situation	Modificateur	Multiplicateur
Contre fort	+30%	× 1.30
Contre léger	+15%	× 1.15
Faible contre	-15%	× 0.85
Mauvais contre	-30%	× 0.70

Boucle pierre-feuille-ciseaux

```
Chasseur / Intercepteur  →  Bombardier
Bombardier               →  Défense au sol / Cuirassé / Capital Ship
Escorte                  →  Ailes légères
Croiseur / Cuirassé      →  Escorte
Capital Ship             →  Croiseur / Cuirassé
Défense au sol           →  Chasseur / Intercepteur / Corvette
```

Défenses statiques

- Les défenses appartiennent à une planète et ne peuvent pas se déplacer.
- Elles sont destructibles. On peut en construire plusieurs de même type sur une planète.

- Dans le resolver de combat, chaque défense est une entité individuelle (comme un vaisseau).
- Construites via la Forteresse (bâtiment dédié).

2. Modèle de données

Le modèle distingue deux natures d'entités :

- Entités statiques (catalogue YAML) : ShipType, BuildingType, DefenseType, Technology — jamais en BDD.
- Entités dynamiques (BDD) : état du jeu, possessions du joueur, files d'attente, flottes, etc.

2.1 Univers et carte

```
Universe
└── Galaxy
    └── SolarSystem
        └── Planet
            ├── position: int (1-9)          # distance à l'étoile
            ├── status: available | colonized
            ├── owner: Player | null
            ├── buildings: PlanetBuilding[]
            ├── defenses: PlanetDefense[]
            └── resources: PlanetResource[]

PlayerDiscovery                              # Fog of War
├── player: Player
└── planet: Planet
```

2.2 Joueur

```
Player
├── alliances: Player[]
├── technologies: PlayerTechnology[]
└── fleets: Fleet[]

PlayerTechnology
├── player: Player
├── technology_slug: string                  # ref YAML
└── level: int
```

2.3 Ressources par planète

```
PlanetResource
├── planet: Planet
├── resource_type: metal | energy | antimatter | hydrogen | carbon
```

```
├── current_stock: float
├── production_rate: float                    # recalculé lors d'un upgrade de
mine
└── last_updated_at: datetime
```

2.4 Bâtiments

```
PlanetBuilding
├── planet: Planet
├── building_type_slug: string                # ref YAML
├── level: int
└── upgrade_finish_at: datetime | null        # null = pas en cours
```

2.5 Flottes et vaisseaux

```
Fleet
├── owner: Player
├── home_planet: Planet                       # planète d'origine (retour)
├── current_location: Planet | null           # null = en transit
├── destination: Planet | null
├── status: idle | traveling | returning | in_combat
├── ships: FleetShip[]
└── arrival_at: datetime | null


FleetShip
├── fleet: Fleet
├── ship_type_slug: string                    # ref YAML ex: 'fighter',
'bomber_mk2'
└── quantity: int
```

2.6 Chantier spatial (file de construction)

```
Shipyard                                      # 1 max par planète
├── planet: Planet (unique)
├── level: int
└── build_queue: ShipyardQueue[]


ShipyardQueue
├── shipyard: Shipyard
├── ship_type_slug: string                    # ref YAML
├── quantity: int
├── queued_at: datetime
```

```
└── finish_at: datetime                    # message RabbitMQ pointant ici
```

2.7 Défenses statiques

```
PlanetDefense
├── planet: Planet
├── defense_type_slug: string              # ref YAML
└── quantity: int
                                           # Pas de current_hp persistant :
                                           # HP remis à max après chaque combat
```

2.8 Recherche

```
ResearchQueue
├── planet: Planet
├── technology_slug: string                # ref YAML
├── started_at: datetime
└── finish_at: datetime                    # message RabbitMQ pointant ici
```

> *Contraintes d'unicité à enforcer en BDD et en service : 1 ResearchQueue active par planète, et un même technology_slug ne peut pas apparaître en cours ou en queue sur deux planètes différentes du même joueur en même temps.*

3. Exemples de fichiers YAML

Tous les catalogues sont dans config/game/. Structure des dossiers :

```
config/game/
├── ships/
│   ├── fighter.yaml
│   ├── interceptor.yaml
│   ├── heavy_fighter.yaml
│   ├── stealth_fighter.yaml
│   ├── bomber.yaml
│   ├── corvette.yaml
│   ├── frigate.yaml
│   ├── destroyer.yaml
│   ├── cruiser.yaml
│   ├── battleship.yaml
│   ├── capital.yaml
│   ├── transport.yaml
│   ├── probe.yaml
│   └── colony_ship.yaml
├── buildings/
│   ├── mine_metal.yaml
│   ├── mine_energy.yaml
│   ├── storage_metal.yaml
│   ├── lab.yaml
│   ├── shipyard.yaml
│   └── fortress.yaml
├── technologies/
│   ├── advanced_propulsion.yaml
│   ├── precision_fire.yaml
│   └── ...
├── defenses/
│   ├── ground_turret.yaml
│   └── ...
└── combat_modifiers.yaml
```

3.1 Vaisseau — fighter.yaml

```yaml
# config/game/ships/fighter.yaml
slug: fighter
label: "Chasseur"
category: light

unlock_requirements:
```

```
  technologies: []
  buildings: []
  ships: []

hp: 150
precision: 70
tracking: 80
evasion: 60
signature: 20
base_attack: 40
shots_min: 1
shots_max: 3
build_time_base: 60      # secondes

costs:
  metal: 3000
  energy: 1000
```

3.2 Vaisseau avec prérequis — stealth_fighter.yaml

```
# config/game/ships/stealth_fighter.yaml
slug: stealth_fighter
label: "Chasseur Furtif"
category: light

unlock_requirements:
  technologies:
    - slug: stealth_technology
      level: 2
    - slug: advanced_propulsion
      level: 1
  buildings:
    - slug: shipyard
      level: 4
  ships:
    - slug: fighter          # doit avoir débloqué le chasseur de base

hp: 280
precision: 85
tracking: 75
evasion: 90
signature: 8               # très difficile à toucher
base_attack: 60
```

```
shots_min: 1
shots_max: 2
build_time_base: 240

costs:
  metal: 9000
  energy: 4000
  antimatter: 800
```

3.3 Bâtiment — shipyard.yaml

```
# config/game/buildings/shipyard.yaml
slug: shipyard
label: "Chantier Spatial"
max_level: 10
base_build_time: 1800    # secondes pour construire niveau 1
build_time_scaling: 1.6  # multiplié à chaque niveau

costs_base:
  metal: 10000
  energy: 3000
costs_scaling: 1.8

# Effet sur la construction de vaisseaux :
# build_time_reduction_percent_per_level: 8

levels:
  1:
    unlock_requirements:
      technologies: []
      buildings: []
  3:
    unlock_requirements:
      technologies:
        - slug: advanced_engineering
          level: 1
      buildings:
        - slug: mine_metal
          level: 2
  6:
    unlock_requirements:
      technologies:
        - slug: advanced_engineering
```

```
          level: 3
        - slug: robotics
          level: 1
      buildings:
        - slug: power_plant
          level: 4
  10:
    unlock_requirements:
      technologies:
        - slug: nano_construction
          level: 2
```

3.4 Technologie de déblocage — advanced_propulsion.yaml

```
# config/game/technologies/advanced_propulsion.yaml
slug: advanced_propulsion
label: "Propulsion Avancée"
max_level: 5

unlock_requirements:
  technologies:
    - slug: basic_engineering
      level: 2
  buildings:
    - slug: lab
      level: 1

research_time_base: 3600   # secondes, niveau 1
research_time_scaling: 1.8

costs_base:
  metal: 5000
  energy: 2000
costs_scaling: 2.0

effect:
  type: unlock             # pas de bonus de stat direct
```

3.5 Technologie de performance — precision_fire.yaml

```
# config/game/technologies/precision_fire.yaml
slug: precision_fire
```

```
label: "Tir de Précision"
max_level: 3

unlock_requirements:
  technologies:
    - slug: advanced_targeting
      level: 1

research_time_base: 7200
research_time_scaling: 2.0

costs_base:
  metal: 8000
  antimatter: 1000
costs_scaling: 2.5

effect:
  type: stat_bonus
  targets: all_ships      # all_ships | category:bomber | slug:fighter
  stat: precision
  formula: flat           # flat | percent
  value_per_level: 5      # +5 précision par niveau de recherche
```

3.6 Matrice de combat — combat_modifiers.yaml

```
# config/game/combat_modifiers.yaml
# Lecture : attaquant (ligne) vs défenseur (colonne)
# 1.30 = fort contre | 1.15 = léger avantage
# 0.85 = léger désavantage | 0.70 = mauvais contre

modifiers:
  fighter:
    bomber: 1.30
    interceptor: 1.00
    heavy_fighter: 0.85
    ground_defense: 0.85
    cruiser: 0.70
    battleship: 0.70
    capital: 0.70

  bomber:
    ground_defense: 1.30
    battleship: 1.30
```

```
    capital: 1.30
    cruiser: 1.15
    fighter: 0.70
    interceptor: 0.70
    frigate: 0.85

  capital:
    cruiser: 1.30
    battleship: 1.30
    bomber: 0.70
    fighter: 0.70
    interceptor: 0.70
  # ... compléter pour chaque type de vaisseau
```

4. Code PHP — lecture des catalogues YAML

Les catalogues sont chargés au boot de Symfony via des services injectés partout où c'est nécessaire. Chaque catalogue scanne son dossier entier automatiquement.

4.1 ShipDefinition — objet immuable représentant un type de vaisseau

```
// src/Game/Ship/ShipDefinition.php

final class ShipDefinition
{
    public function __construct(
        public readonly string       $slug,
        public readonly string       $label,
        public readonly ShipCategory $category,
        public readonly int          $hp,
        public readonly int          $precision,
        public readonly int          $tracking,
        public readonly int          $evasion,
        public readonly int          $signature,
        public readonly int          $baseAttack,
        public readonly int          $shotsMin,
        public readonly int          $shotsMax,
        public readonly int          $buildTimeBase,
        public readonly array        $costs,
        public readonly ?string      $unlockRequirements,
    ) {}

    public static function fromArray(string $slug, array $data): self
    {
        return new self(
            slug:              $slug,
            label:             $data['label'],
            category:          ShipCategory::from($data['category']),
            hp:                $data['hp'],
            precision:         $data['precision'],
            tracking:          $data['tracking'],
            evasion:           $data['evasion'],
            signature:         $data['signature'],
            baseAttack:        $data['base_attack'],
            shotsMin:          $data['shots_min'],
            shotsMax:          $data['shots_max'],
            buildTimeBase:     $data['build_time_base'],
```

```
            costs:              $data['costs'],
            unlockRequirements: $data['unlock_requirements'] ?? null,
        );
    }
}
```

4.2 ShipCatalog — service Symfony chargé au boot

```
// src/Game/Catalog/ShipCatalog.php

class ShipCatalog
{
    /** @var array<string, ShipDefinition> */
    private array $ships = [];

    public function __construct(string $configDir)
    {
        // Scan du dossier — 1 fichier YAML par vaisseau
        foreach (glob($configDir . '/ships/*.yaml') as $file) {
            $data = Yaml::parseFile($file);
            $slug = $data['slug'];
            $this->ships[$slug] = ShipDefinition::fromArray($slug, $data);
        }
    }

    public function get(string $slug): ShipDefinition
    {
        if (!isset($this->ships[$slug])) {
            throw new \InvalidArgumentException("Ship type '$slug' not found");
        }
        return $this->ships[$slug];
    }

    /** @return ShipDefinition[] */
    public function all(): array
    {
        return $this->ships;
    }
}
```

Le service est déclaré dans services.yaml avec le chemin du dossier en argument :

```
# config/services.yaml
```

```
App\Game\Catalog\ShipCatalog:
    arguments:
        $configDir: '%kernel.project_dir%/config/game'
```

4.3 RequirementChecker — vérification unifiée des prérequis

```
// src/Game/Requirement/RequirementChecker.php

class RequirementChecker
{
    public function __construct(
        private readonly ShipCatalog $shipCatalog,
    ) {}

    public function check(
        array  $requirements,
        Player $player,
        Planet $planet,
    ): RequirementResult {
        $failures = [];

        // Vérification des technologies (niveau global joueur)
        foreach ($requirements['technologies'] ?? [] as $req) {
            $playerLevel = $player->getTechnologyLevel($req['slug']);
            if ($playerLevel < $req['level']) {
                $failures[] = new MissingTechnology($req['slug'], $req['level'],
$playerLevel);
            }
        }

        // Vérification des bâtiments (niveau local planète)
        foreach ($requirements['buildings'] ?? [] as $req) {
            $planetLevel = $planet->getBuildingLevel($req['slug']);
            if ($planetLevel < $req['level']) {
                $failures[] = new MissingBuilding($req['slug'], $req['level'],
$planetLevel);
            }
        }

        // Vérification des vaisseaux prérequis (récursif)
        foreach ($requirements['ships'] ?? [] as $shipSlug) {
            if (!$this->isShipUnlocked($shipSlug, $player, $planet)) {
                $failures[] = new MissingShipUnlock($shipSlug);
            }
```

```
        }

        return new RequirementResult($failures);
    }

    private function isShipUnlocked(
        string $shipSlug,
        Player $player,
        Planet $planet,
        array  $visited = [],
    ): bool {
        // Protection contre les cycles dans les prérequis
        if (in_array($shipSlug, $visited, true)) {
            throw new \LogicException(
                'Cycle dans les prérequis : ' . implode(' -> ', $visited) . ' ->
' . $shipSlug
            );
        }

        $visited[]   = $shipSlug;
        $definition  = $this->shipCatalog->get($shipSlug);
        $requirements = $definition->unlockRequirements ?? [];

        $result = $this->check($requirements, $player, $planet);
        if (!$result->isSatisfied()) {
            return false;
        }

        // Récursion sur les vaisseaux prérequis
        foreach ($requirements['ships'] ?? [] as $requiredSlug) {
            if (!$this->isShipUnlocked($requiredSlug, $player, $planet,
$visited)) {
                return false;
            }
        }

        return true;
    }
}
```

4.4 TechnologyBonusResolver — calcul des bonus actifs au moment du combat

```
// src/Game/Combat/TechnologyBonusResolver.php

class TechnologyBonusResolver
{
    public function __construct(
        private readonly TechnologyCatalog $techCatalog,
        private readonly ShipCatalog       $shipCatalog,
    ) {}

    /**
     * Retourne tous les bonus actifs du joueur au moment de l'appel.
     * Appelé une seule fois au début du calcul de combat.
     */
    public function resolve(Player $player): ShipBonusCollection
    {
        $bonuses = new ShipBonusCollection();

        foreach ($player->getResearchedTechnologies() as $playerTech) {
            $definition = $this->techCatalog->get($playerTech->getTechnologySlug());
            $effect     = $definition->getEffect();

            if ($effect['type'] !== 'stat_bonus') {
                continue;
            }

            // Calcul de la valeur selon le niveau actuel
            $value = $effect['value_per_level'] * $playerTech->getLevel();

            // Application sur les slugs cibles
            foreach ($this->resolveTargets($effect['targets']) as $shipSlug) {
                $bonuses->add($shipSlug, $effect['stat'], $value, $effect['formula']);
            }
        }

        return $bonuses;
    }

    /** @return string[] liste de slugs de vaisseaux concernés */
    private function resolveTargets(string $targets): array
    {
        if ($targets === 'all_ships') {
            return array_keys($this->shipCatalog->all());
```

```
        }

        // Format : 'category:bomber' ou 'slug:fighter'
        [$type, $value] = explode(':', $targets, 2);

        return match ($type) {
            'category' => array_keys(array_filter(
                $this->shipCatalog->all(),
                fn(ShipDefinition $s) => $s->category->value === $value,
            )),
            'slug' => [$value],
            default => throw new \InvalidArgumentException("Cible inconnue :
$targets"),
        };
    }
}
```

4.5 Résumé du flux — construction d'un vaisseau

```
// Exemple : le joueur veut construire 5 stealth_fighter sur sa planète

// 1. Récupérer la définition depuis le catalogue
$definition = $shipCatalog->get('stealth_fighter');

// 2. Vérifier les prérequis
$result = $requirementChecker->check(
    $definition->unlockRequirements,
    $player,
    $planet,
);

if (!$result->isSatisfied()) {
    // Renvoyer les prérequis manquants à l'UI
    return $result->getFailures();
}

// 3. Vérifier les ressources disponibles (lazy calculation)
$cost = $definition->costs;   // ['metal' => 9000, 'energy' => 4000, ...]
$totalCost = array_map(fn($v) => $v * 5, $cost);   // x5 vaisseaux
$planetResourceService->consume($planet, $totalCost);

// 4. Calculer le temps de construction (avec bonus chantier)
$shipyard   = $planet->getShipyard();
```

```
$buildTime  = $buildTimeCalculator->calculate($definition, $shipyard->getLevel(),
5);


// 5. Créer l'entrée en queue et émettre le message RabbitMQ
$queue = new ShipyardQueue(
    shipyard:      $shipyard,
    shipTypeSlug:  'stealth_fighter',
    quantity:      5,
    queuedAt:      new \DateTimeImmutable(),
    finishAt:      new \DateTimeImmutable("+ {$buildTime} seconds"),
);
$entityManager->persist($queue);
$messageBus->dispatch(new ShipConstructionFinished($queue->getId()), [
    new DelayStamp($buildTime * 1000),  // millisecondes
]);
```

> *Le consumer RabbitMQ qui traite ShipConstructionFinished ajoute les vaisseaux à la FleetShip de la planète (ou crée une nouvelle ligne si le slug n'existait pas encore) et notifie le joueur via Mercure.*